IF2110 Algoritma dan Pemrograman 2

Type Rekursif List
LIST OF LIST

Tim Pengajar IF2110

Sekolah Teknik Elektro dan Informatika

Definisi rekursif

List of list adalah list yang:

- mungkin kosong
- mungkin terdiri dari sebuah elemen yang disebut **atom dan sisanya adalah list of list**
- mungkin terdiri dari sebuah elemen berupa **list dan sisanya adalah list of list**

→List of List adalah list S yang elemennya adalah list

Contoh List

- [ ] adalah list kosong
- [ad , a , b] adalah list dengan elemen berupa 3 atom
- [[ ] , [a , b , c] , [d , [e , f]] , g] adalah :
  - list dengan elemen list kosong, S1, S2, dan sebuah atom g
    - S1 adalah list dengan elemen 3 buah atom a, b, c
    - S2 adalah list dengan elemen sebuah atom d dan S3
      - S3 adalah list dengan elemen 2 buah atom e, f

Type List of List

Type List of List

DEFINISI DAN SPESIFIKASI PREDIKAT KHUSUS UNTUK LIST OF LIST

IsEmpty: list of list → boolean

{IsEmpty(S) benar jika S adalah list of list kosong}

IsOneElmt: list of list → boolean

{IsOneElmt(S) benar jika S adalah list of list dengan 1 elemen}

IsAtom: Atom/List → boolean

{IsAtom(S) menghasilkan true jika S merupakan sebuah atom}

IsList: Atom/List → boolean

{IsList(S) menghasilkan true jika S adalah sebuah list (bukan atom)}

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

KonsLo: Atom/List, list of list → list of list

{KonsLo(L,S) diberikan sebuah Atom/List L dan sebuah List of List S, membentuk list baru dengan L sebagai elemen pertama: L o S → S'}

KonsL•: list of list, Atom/List → list of list

{KonsL•(S,L) diberikan sebuah List of List S dan sebuah Atom/List L, membentuk list baru dengan L sebagai elemen terakhir: S • L → S'}

Type List of List (2)

DEFINISI DAN SPESIFIKASI SELEKTOR

FirstList: list of list tidak kosong → Atom/List
{FirstList(S) Menghasilkan elemen pertama list, mungkin sebuah list atau atom}

TailList : list of list tidak kosong → list of list
{TailList(S) Menghasilkan "sisa" list of list S tanpa elemen pertama list S}

LastList : list of list tidak kosong → Atom/List
{LastList(S) Menghasilkan elemen terakhir list of list S, mungkin list atau atom}

HeadList : list of list tidak kosong → list of list
{HeadList(S) Menghasilkan "sisa" list of list tanpa elemen terakhir list S}

Apakah dua buah list of list sama

- IsEqS: 2 list of list → boolean
- Contoh:
  - IsEqS([ ],[ ])=true
  - IsEqS([ ],[a])=false
  - IsEqS([a],[ ])=false
  - IsEqS([[a,b],c],[[a,b],c])=true
- Analisis Rekurens
  - Basis: Jika dua2nya kosong maka true, jika hanya salah satu kosong maka false
  - Rekurens: Cek elemen pertama dari kedua list dan kemudian cek tail kedua list tersebut ATAU cek elemen terakhir dari kedua list dan kemudian cek head kedua list

IsEqS – Realisasi

IsEqS(S1,S2):

depend on S1,S2

IsEmpty(S1) and IsEmpty(S2): true {basis}

IsEmpty(S1) and not isEmpty(S2): false {basis}

not IsEmpty(S1) and isEmpty(S2): false {basis}

not IsEmpty(S1) and not isEmpty(S2): {rekurens}

depend on (FirstList(S1), FirstList(S2))

IsAtom(FirstList(S1)) and IsAtom(FirstList(S2)):
FirstList(S1) = FirstList(S2) and then
IsEqS(TailList(S1),TailList(S2))

IsList(FirstList(S1)) and IsList(FirstList(S2)):
IsEqS(FirstList(S1),FirstList(S2)) and
IsEqS(TailList(S1),TailList(S2))

else : {atom dan list pasti tidak sama}
false

Apakah sebuah **atom** merupakan elemen list of list

- IsMemberS: elemen, list of list → boolean
- Rekursif:
  - Basis 0: list kosong → false
  - Rekurens: cek apakah ada pada elemen pertama atau pada tail dari list tersebut
- Realisasi:

```
IsMemberS(A,S) :
  if IsEmpty(S) then false {basis 0}
  else {rekurens}
    if IsAtom(FirstList(S)) then
      A = FirstList(S) or else IsMemberS(A,TailList(S))
    else {FirstList(S) adalah list}
      IsMemberS(A,FirstList(S)) or IsMemberS(A,TailList(S))
```

Menghapus elemen (atom) dari list of list

- Rember*: elemen, list of list → list of list
- Contoh:
  - Rember*(a,[]) = []
  - Rember*(a,[[b,c],a,d,[b,c,a]]) = [[b,c],d,[b,c]]
- Rekursif
  - Basis 0: Jika list kosong → []
  - Rekurens: Cek elemen pertama list:
    - Jika atom = a → tail list tanpa a
    - Jika atom ≠ a → (first elemen) o (tail list tanpa a)
    - Jika list → (first elemen tanpa a) o (tail list tanpa a)

Rember* – Realisasi

```
Rember*(a,S) :
  if IsEmpty(S)  then S {Basis}
  else {Rekurens}
      if IsList(FirstList(S))  then
          KonsLo (Rember*(a,FirstList(S)),
                  Rember*(a,TailList(S)))
      else { elemen pertama S adalah atom }
          if FirstList(S) = a then
              Rember*(a,TailList(S))
          else
              KonsLo(FirstList(S),Rember*(a,TailList(S)))
```

Maksimum dari list of list dengan elemen integer

- Max: list of list tidak kosong → <u>integer</u>
- Catatan: elemen berbentuk list di dalam list of list juga tidak boleh kosong
- Rekursif
  - Basis 1:
    - Jika elemen adalah atom → elemen
    - Jika elemen adalah list → Max(elemen)
  - Rekurens: Cek elemen pertama list:
    - Jika atom → Max2(elemen,Max(tail))
    - Jika list → Max2(Max(elemen),Max(tail))

Max – Realisasi

```
Max(S) :
  if IsOneElmt(S)  then  {Basis 1 – tidak murni}
      if IsAtom(FirstList(S))  then
            FirstList(S)
      else   { elemen pertama adalah list }
            Max(FirstList(S))  {sudah bagian rekurens}
  else  {Rekurens}
      if IsAtom(FirstList(S))  then {First elemen adl. atom }
            Max2(FirstList(S),Max(TailList(S))
      else   { First element adalah List }
            Max2(Max(FirstList(S)), Max(TailList(S))
```

Resume Analisis Rekurens terhadap LIST

Rekurens terhadap List: f(L)

- Basis kosong:
  - Basis: IsEmpty(L): ekspresi basis
  - Rekurens: f(Tail(L))
- Basis satu:
  - Basis: IsOneElmt(L): ekspresi basis
  - Rekurens: f(Tail(L)) {minimal 2 elemen}

Rekurens terhadap List of List: f(S)

- Basis: IsEmpty(S): ekspresi basis
- Rekurens:

  <u>depend on</u> FirstList(S)

  IsAtom(FirstList(S)):

  g(ekspresi terhadap atom, f(TailList(S)))

  IsList(FirstList(S)):

  g(ekspresi terhadap list, f(TailList(S)))

Sumber

- Diktat “Dasar Pemrograman, Bag. Pemrograman Fungsional” oleh Inggriani Liem, revisi Februari 2014